

MLIS

Prof. (Dr.) Anjali Diwan
Computer Science & Engineering



Session Goals: Building

1. Understand real industry expectations
2. Learn structured professional introduction
3. Build a strong placement strategy
4. Create a high-impact resume
5. Establish a credible online presence

Different Learning Methods

- Eager Learning

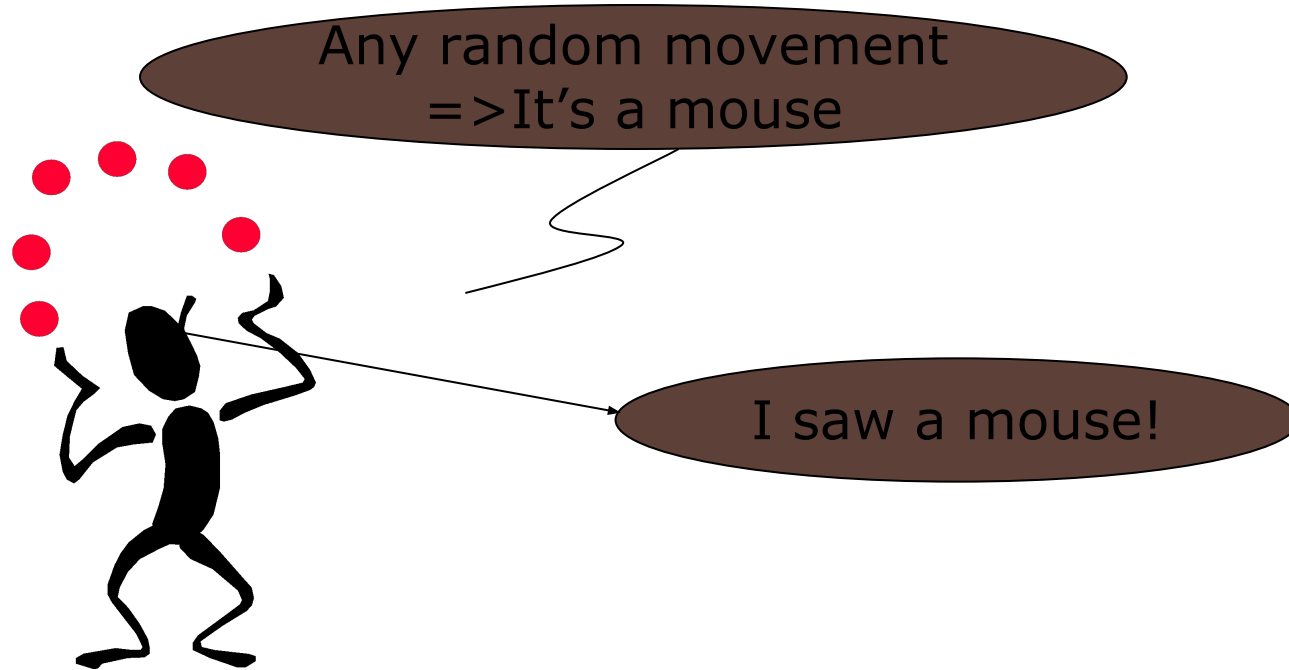
- Explicit description of target function on the whole training set

- Instance-based Learning

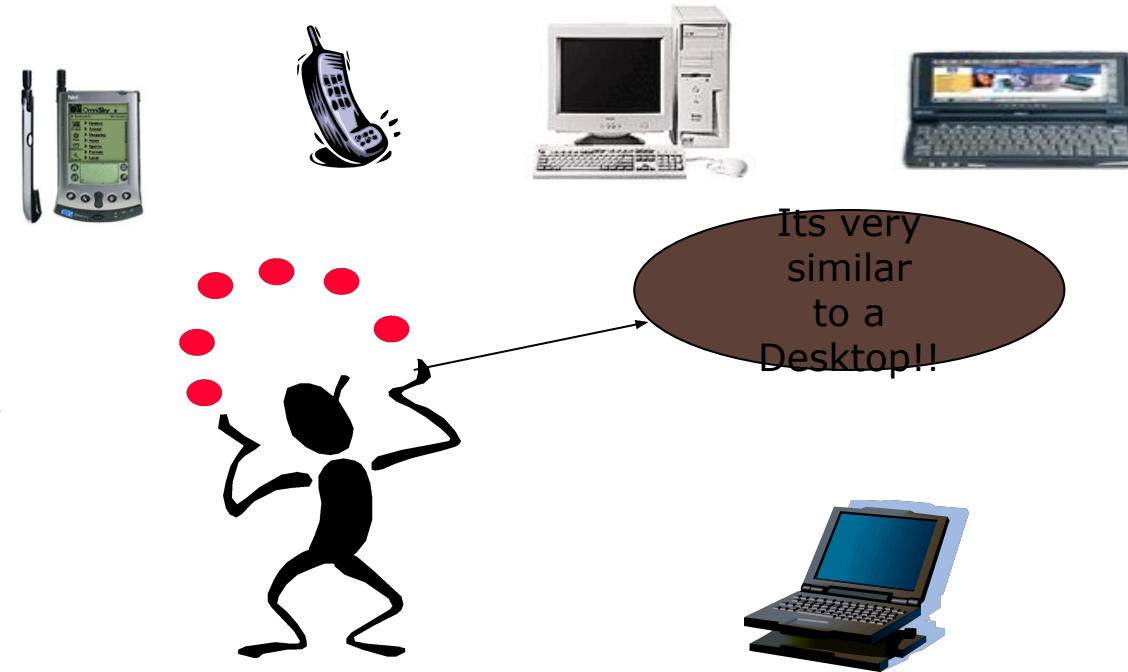
- Learning=storing all training instances
 - Classification=assigning target function to a new instance
 - Referred to as “Lazy” learning

Different Learning Methods

Eager Learning



Instance-based Learning

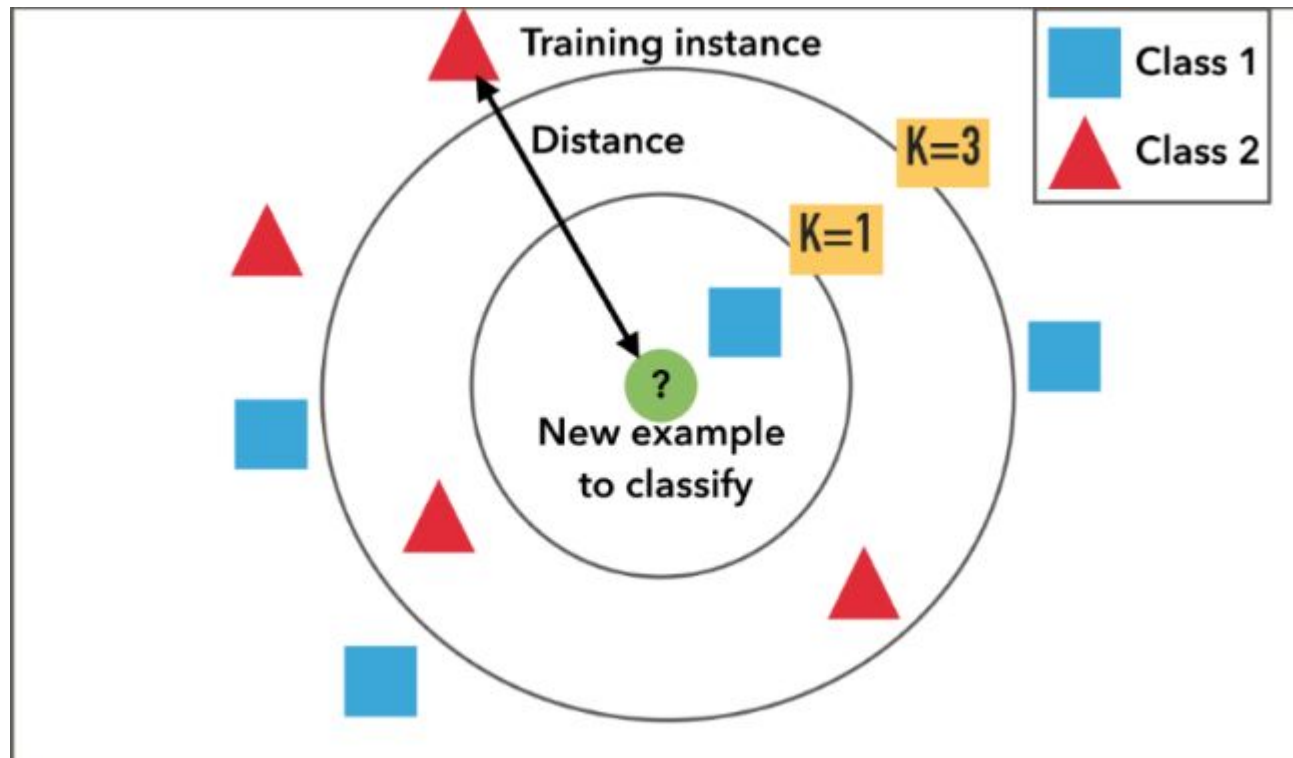


Instance-based Learning

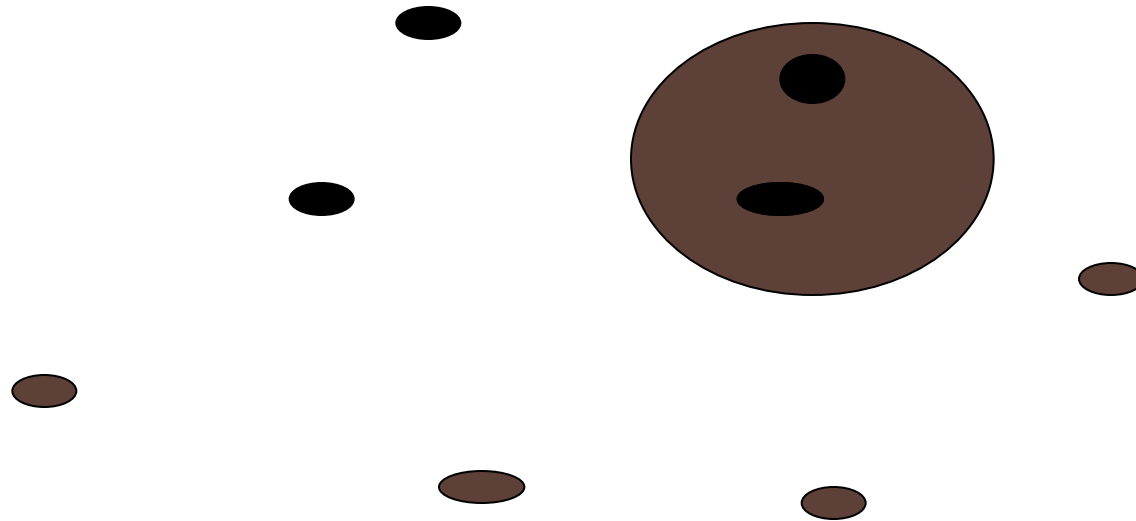
- K-Nearest Neighbor Algorithm
- Weighted Regression
- Case-based reasoning

- KNN algorithm is one of the simplest classification algorithm
- non-parametric
 - it does not make any assumptions on the underlying data distribution
- lazy learning algorithm.
 - there is *no explicit training phase* or it is very minimal.
 - also means that the training phase is pretty fast .
 - Lack of generalization means that KNN keeps all the training data.
- Its purpose is to use a database in which the data points are separated into several classes to predict the classification of a new sample point.

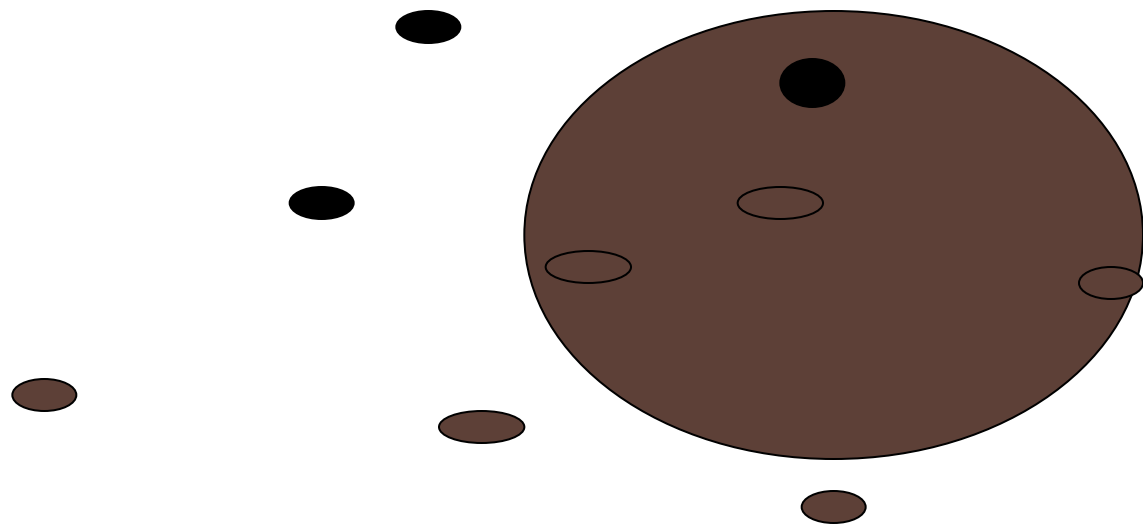
- KNN Algorithm is based on feature similarity
- How closely out-of-sample features resemble our training set determines how we classify a given data point



1-Nearest Neighbor



3-Nearest Neighbor



Classification steps

1. Training phase: a model is constructed from the training instances.
 - classification algorithm finds relationships between predictors and targets
 - relationships are summarised in a model
2. Testing phase: test the model on a test sample whose class labels are known but not used for training the model
3. Usage phase: use the model for classification on new data whose class labels are unknown

K-Nearest Neighbor

■ Features

- All instances correspond to points in an n -dimensional Euclidean space
- Classification is delayed till a new instance arrives
- Classification done by comparing feature vectors of the different points
- Target function may be discrete or real-valued

K-Nearest Neighbor

- An arbitrary instance is represented by $(a_1(x), a_2(x), a_3(x), \dots, a_n(x))$
 - $a_i(x)$ denotes features
- Euclidean distance between two instances
$$d(x_i, x_j) = \sqrt{\sum_{r=1}^n (a_r(x_i) - a_r(x_j))^2}$$
- Continuous valued target function
 - mean value of the k nearest training examples

- K-nearest neighbours uses the local neighborhood to obtain a prediction
- The K memorized examples more similar to the one that is being classified are retrieved
- A distance function is needed to compare the examples similarity
- This means that if we change the distance function, we change how examples are classified

Euclidean :

$$d(x, y) = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

Manhattan / city - block :

$$d(x, y) = \sum_{i=1}^m |x_i - y_i|$$

- If the ranges of the features differ, features with bigger values will dominate decision
- In general feature values are normalized prior to distance calculation

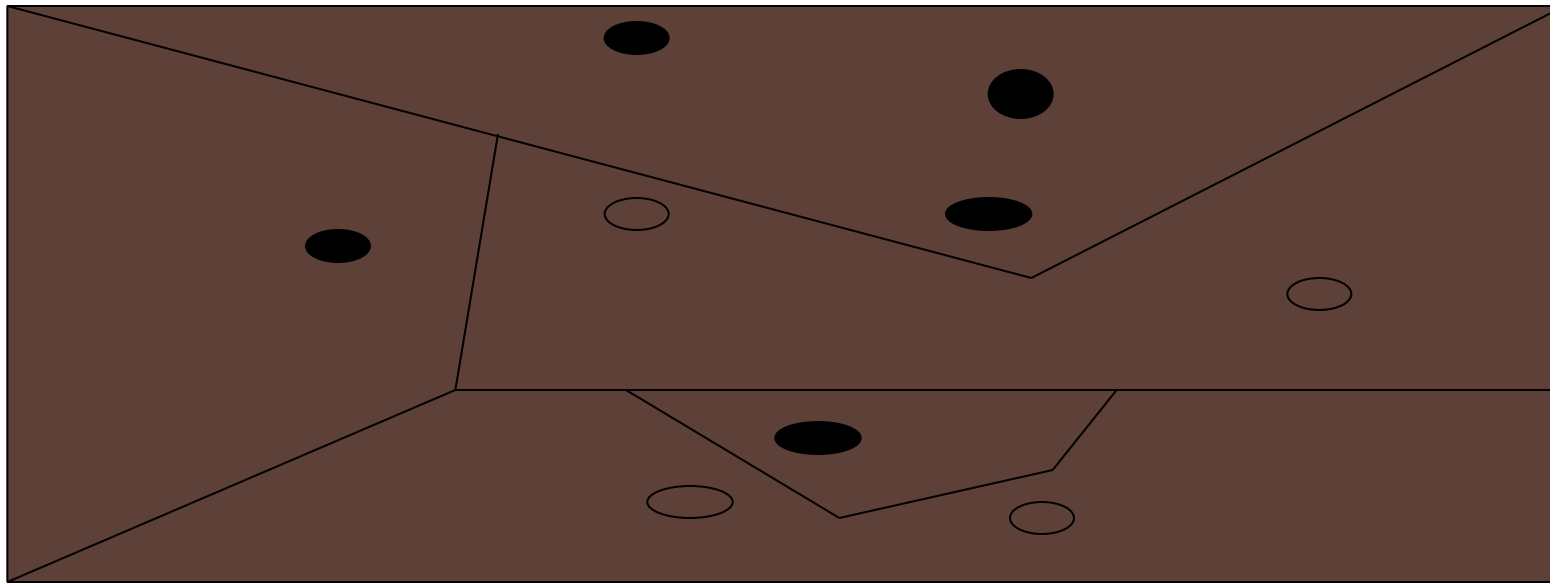
$$X_s = \frac{X - \text{mean}}{s.d.}$$

$$X_s = \frac{X - \text{mean}}{\text{max} - \text{min}}$$

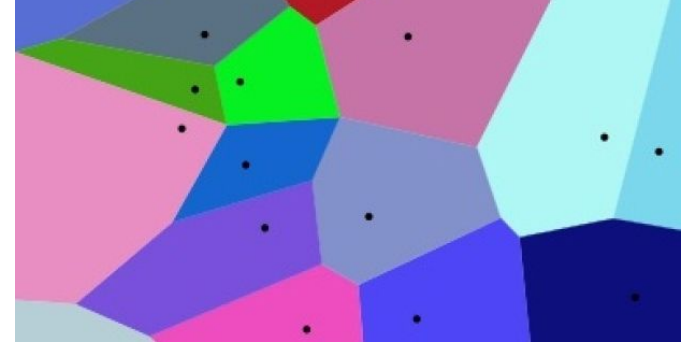
$$X_s = \frac{X - \text{min}}{\text{max} - \text{min}}$$

Voronoi Diagram

- Decision surface formed by the training examples



Voronoi diagram



- We frequently need to find the nearest hospital, surgery or supermarket.
- A map divided into cells, each cell covering the region closest to a particular centre, can assist us in our quest.

Distance-Weighted Nearest Neighbor Algorithm

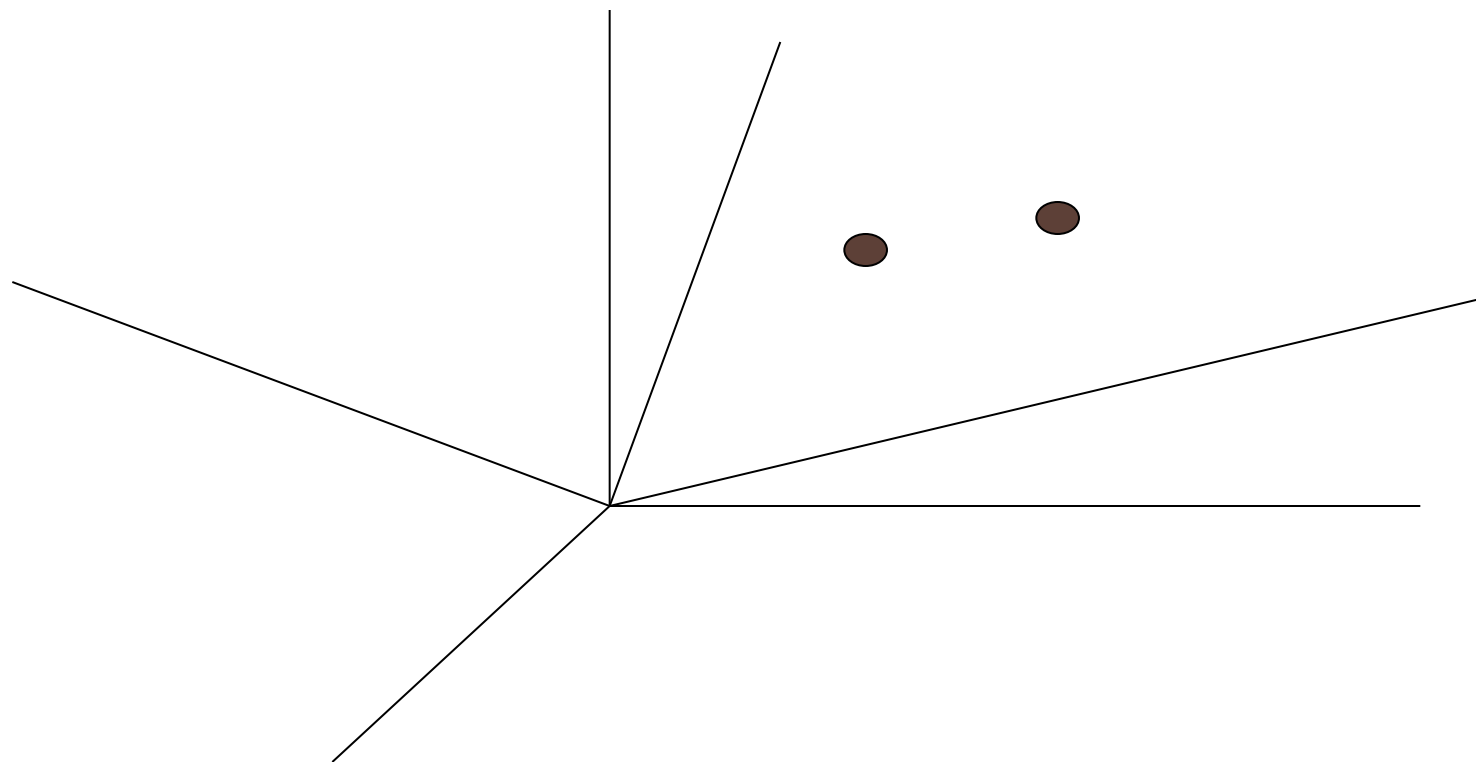
- Assign weights to the neighbors based on their 'distance' to the query point
 - Weight 'may' be inverse square of the distances
- All training points may influence a particular instance
 - Shepard's method

Remarks

- + Highly effective inductive inference method for noisy training data and complex target functions
- + Target function for a whole space may be described as a combination of less complex local approximations
- + Learning is very simple
- Classification is time consuming

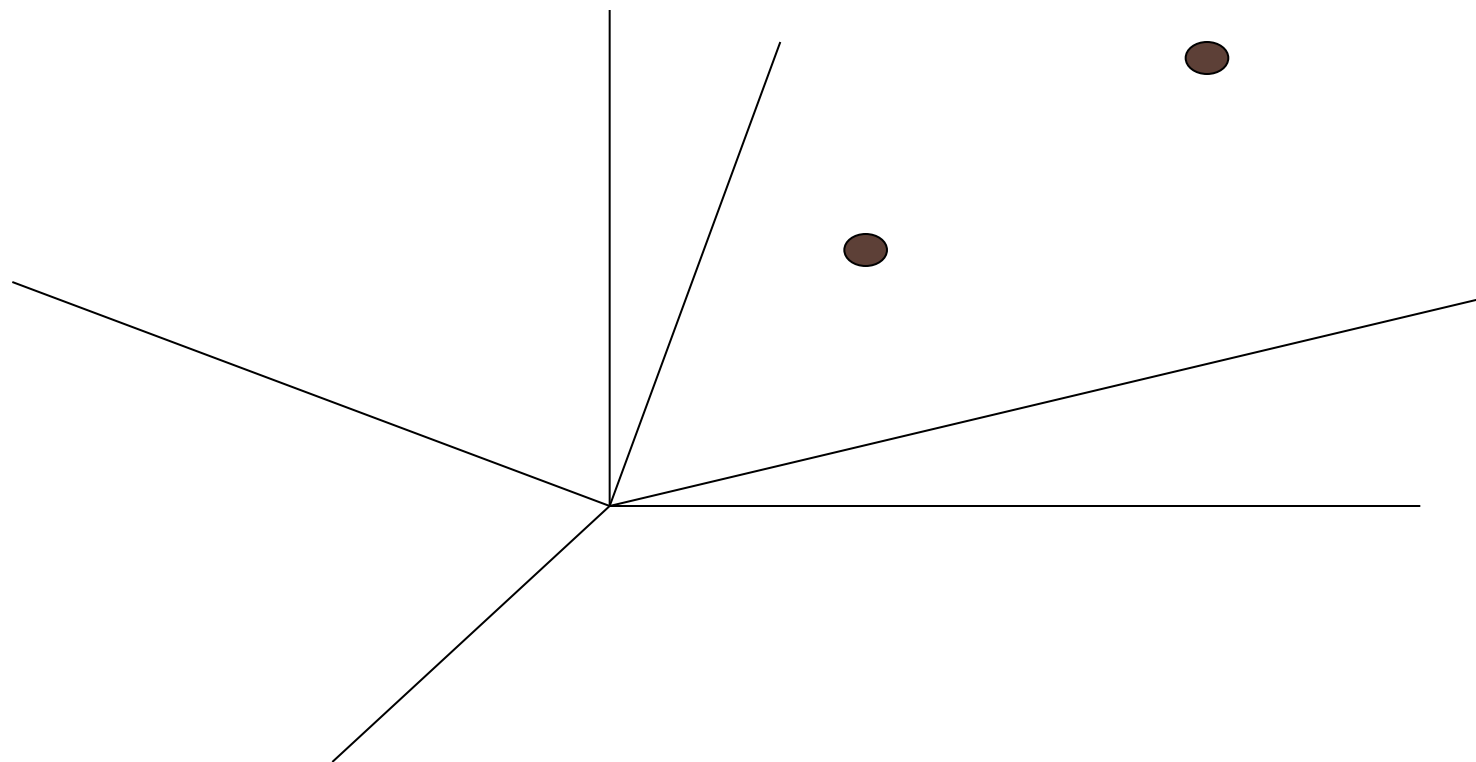
Remarks

- Curse of Dimensionality



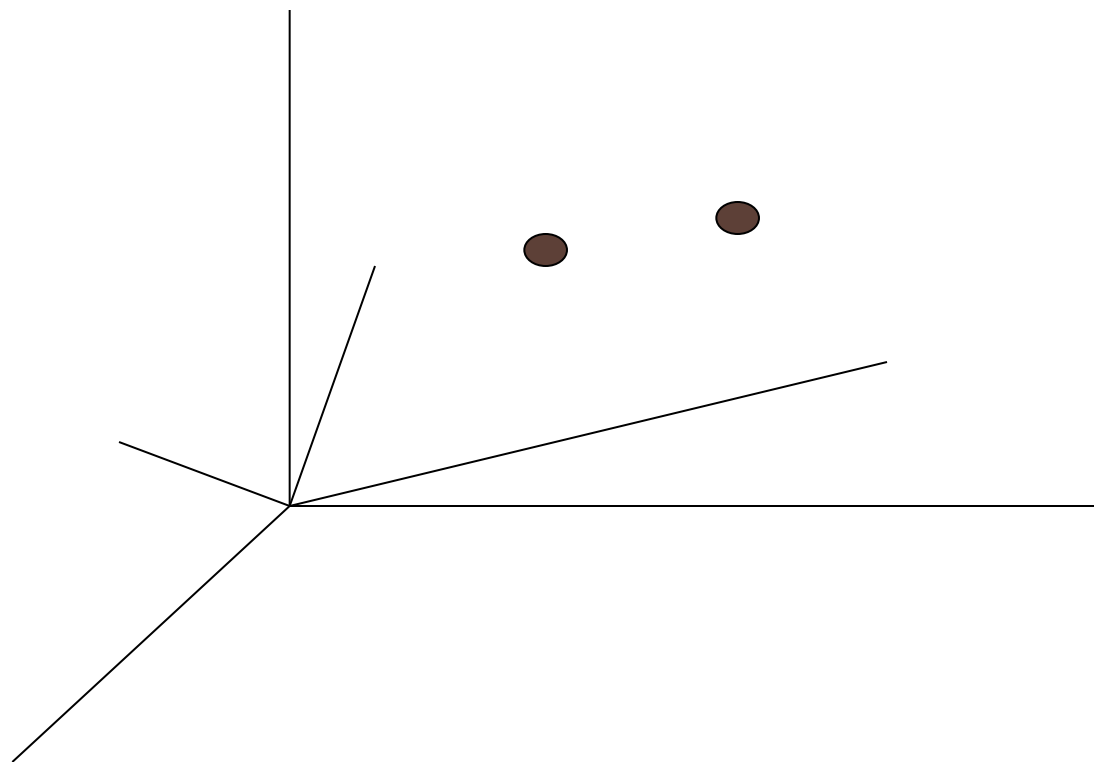
Remarks

- Curse of Dimensionality



Remarks

- Curse of Dimensionality



Nearest-Neighbor Classifiers: Issues

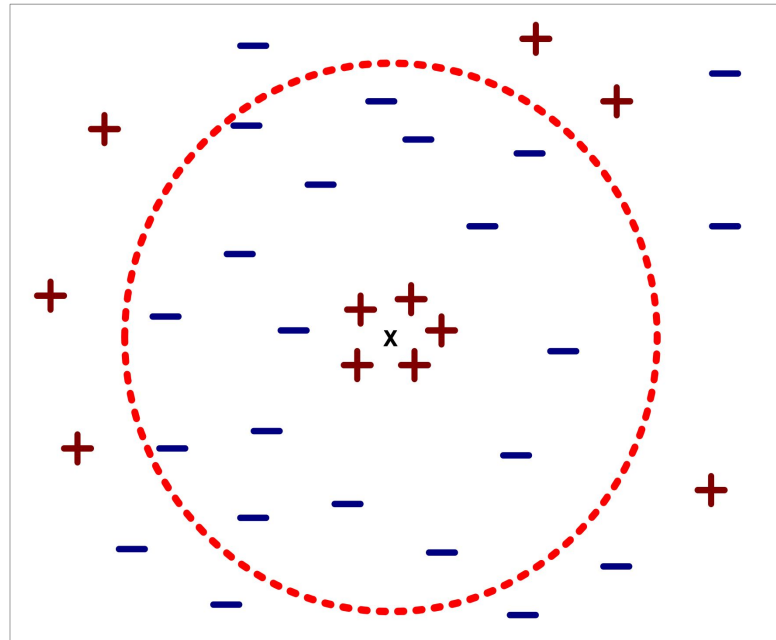
- The value of k , the number of nearest neighbors to retrieve
- Choice of Distance Metric to compute distance between records
- Computational complexity
 - Size of training set
 - Dimension of data



Value of K

- Choosing the value of k:
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes

Rule of thumb:
 $K = \sqrt{N}$
N: number of training points



Distance Metrics

Minkowsky:

$$D(x, y) = \left(\sum_{i=1}^m |x_i - y_i|^r \right)^{1/r}$$

Euclidean:

$$D(x, y) = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

Manhattan / city-block:

$$D(x, y) = \sum_{i=1}^m |x_i - y_i|$$

Camberra:

$$D(x, y) = \sum_{i=1}^m \frac{|x_i - y_i|}{|x_i + y_i|}$$

Chebychev:

$$D(x, y) = \max_{i=1}^m |x_i - y_i|$$

Quadratic:

$$D(x, y) = (x - y)^T Q (x - y) = \sum_{j=1}^m \left(\sum_{i=1}^m (x_i - y_i) q_{ji} \right) (x_j - y_j)$$

Q is a problem-specific positive definite $m \times m$ weight matrix

Mahalanobis:

$$D(x, y) = [\det V]^{1/m} (x - y)^T V^{-1} (x - y)$$

V is the covariance matrix of $A_1..A_m$, and A_j is the vector of values for attribute j occurring in the training set instances 1.. n .

Correlation:

$$D(x, y) = \frac{\sum_{i=1}^m (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^m (x_i - \bar{x})^2 \sum_{i=1}^m (y_i - \bar{y})^2}}$$

$\bar{x}_i = \bar{y}_i$ and is the average value for attribute i occurring in the training set.

Chi-square:

$$D(x, y) = \sum_{i=1}^m \frac{1}{sum_i} \left(\frac{x_i}{size_x} - \frac{y_i}{size_y} \right)^2$$

sum_i is the sum of all values for attribute i occurring in the training set, and $size_x$ is the sum of all values in the vector x .

Kendall's Rank Correlation:

$$D(x, y) = 1 - \frac{2}{n(n-1)} \sum_{i=1}^m \sum_{j=1}^{i-1} \text{sign}(x_i - x_j) \text{sign}(y_i - y_j)$$

$\text{sign}(x) = -1, 0$ or 1 if $x < 0$, $x = 0$, or $x > 0$, respectively.

Figure 1. Equations of selected distance functions.
(x and y are vectors of m attribute values).

Distance Measure: Scale Effects

- Different features may have different measurement scales
 - E.g., patient weight in kg (range [50,200]) vs. blood protein values in ng/dL (range [-3,3])
- Consequences
 - Patient weight will have a much greater influence on the distance between samples
 - May bias the performance of the classifier



Nearest Neighbor : Dimensionality

□ Problem with Euclidean measure:

- High dimensional data
 - **curse of dimensionality**
- Can produce counter-intuitive results
- Shrinking density – sparsification effect

| | | | | | | | | | | 0

0 | | | | | | | | | | |

$d = 1.4142$

vs

| 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 1

$d = 1.4142$



Distance for Nominal Attributes

Value Difference Metric (VDM)

[Stanfill & Waltz, 1986]

Providing appropriate distance measurements for nominal attributes.

$$vdm_a(x, y) = \sum_{c=1}^C \left(\frac{N_{a,x,c}}{N_{a,x}} - \frac{N_{a,y,c}}{N_{a,y}} \right)^2$$

$N_{a,x}$ = # times attribute a had value x

$N_{a,x,c}$ = # times attribute a had value x and class was c

C = # output classes

Two values are considered closer if they have more similar classifications, i.e., if they have more similar correlations with the output classes.

Distance for Heterogeneous Data

In this section, we define a heterogeneous distance function *HVDM* that returns the distance between two input vectors $\mathbf{x}$ and $\mathbf{y}$. It is defined as follows:

$$HVDM(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{a=1}^m d_a^2(x_a, y_a)} \quad (11)$$

where m is the number of attributes. The function $d_a(x, y)$ returns a distance between the two values x and y for attribute a and is defined as:

$$d_a(x, y) = \begin{cases} 1, & \text{if } x \text{ or } y \text{ is unknown; otherwise...} \\ \text{normalized_vdm}_a(x, y), & \text{if } a \text{ is nominal} \\ \text{normalized_diff}_a(x, y), & \text{if } a \text{ is linear} \end{cases} \quad (12)$$

Wilson, D. R. and Martinez, T. R., Improved Heterogeneous Distance Functions, Journal of Artificial Intelligence Research, vol. 6, no. 1, pp. 1-34, 1997



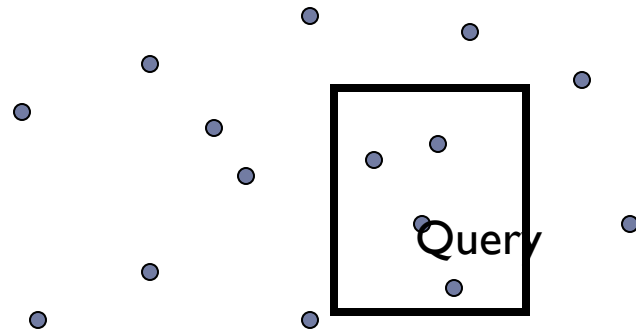
Nearest Neighbour : Computational Complexity

- Expensive
 - To determine the nearest neighbour of a query point q , must compute the distance to all N training examples
 - + Pre-sort training examples into fast data structures (kd-trees)
 - + Compute only an approximate distance (LSH)
 - + Remove redundant data (condensing)
- Storage Requirements
 - Must store all training data **P**
 - + Remove redundant data (condensing)
 - Pre-sorting often increases the storage requirements
- High Dimensional Data
 - “Curse of Dimensionality”
 - Required amount of training data increases exponentially with dimension
 - Computational cost also increases dramatically
 - Partitioning techniques degrade to linear search in high dimension



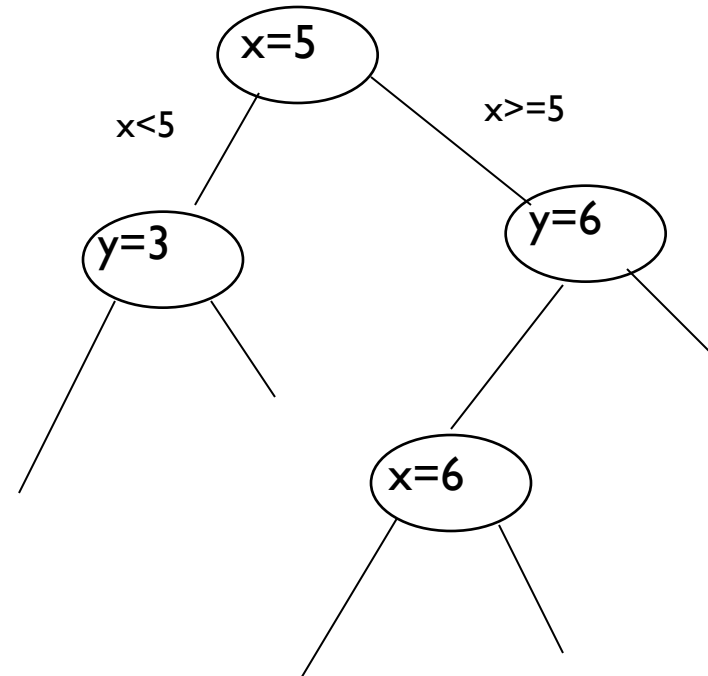
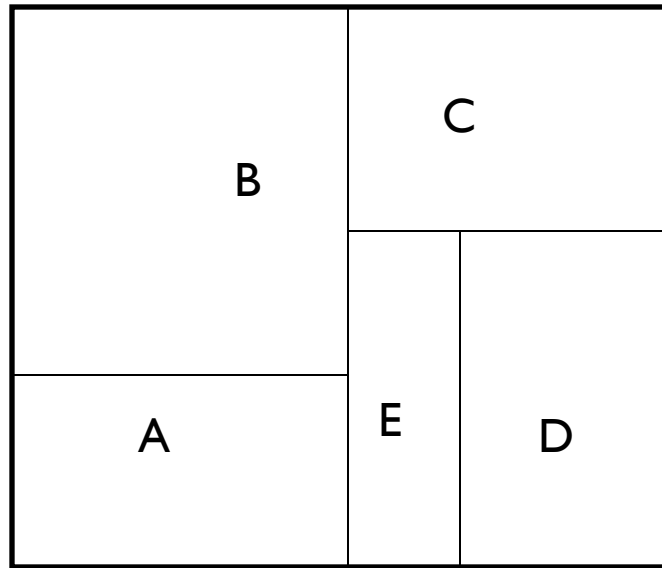
High dimensional search

- Given a point set and a nearest neighbor query point
- Find the points enclosed in a rectangle (range) around the query
- Perform linear search for nearest neighbor only in the rectangle

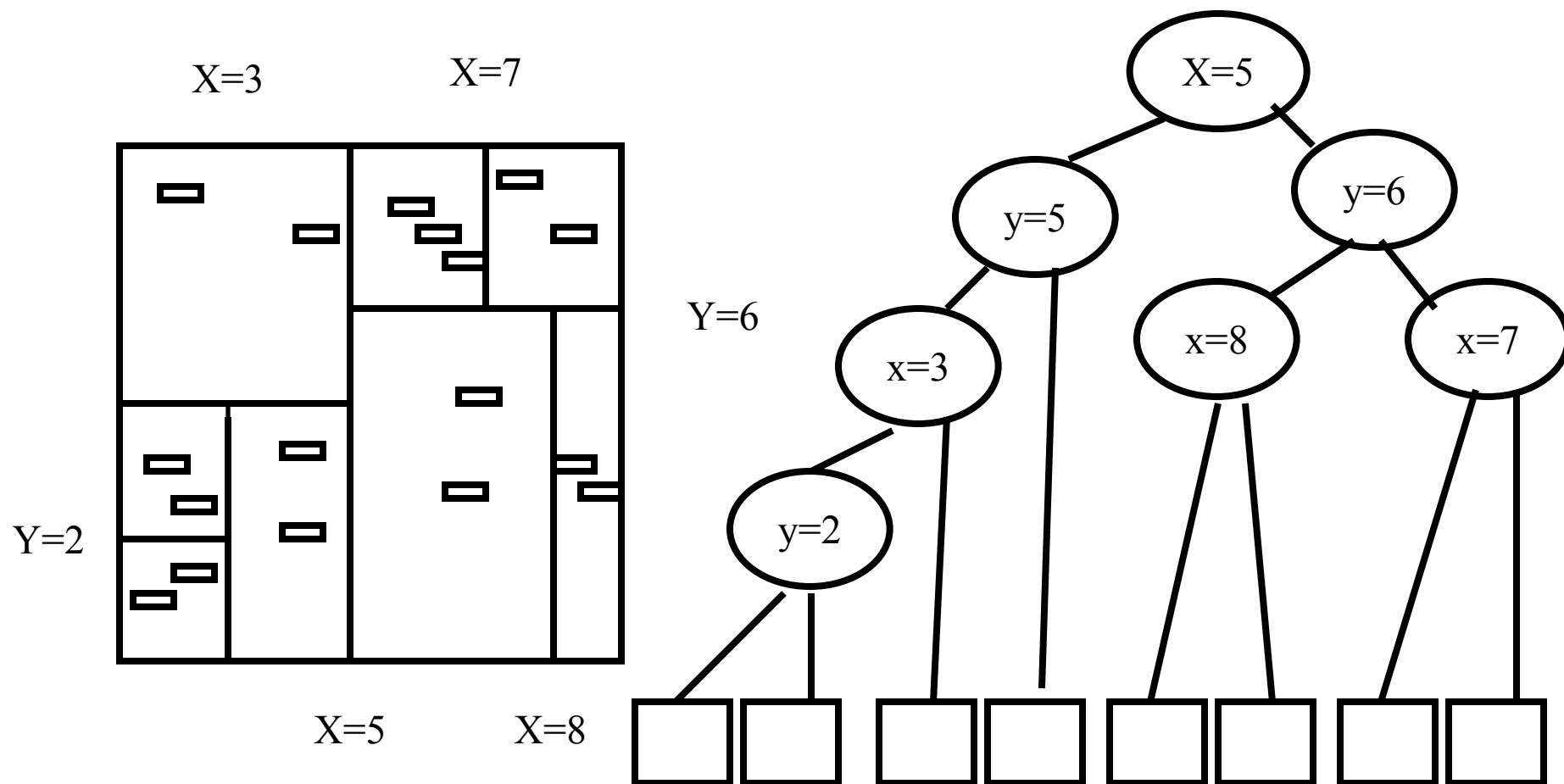


kd-tree: data structure for range search

- Index data into a tree
- Search on the tree
- Tree construction: At each level we use a different dimension to split



kd-tree example



KNN: Alternate Terminologies

- ☐ Instance Based Learning
- ☐ Lazy Learning
- ☐ Case Based Reasoning
- ☐ Exemplar Based Learning



Locally Weighted Regression

Locally Weighted Regression (LWR/LOESS) is a non parametric instance-based learning Algorithm that avoids predefined global models.

There is no explicit training phase. All computation, such as finding neighbors and building local models, occurs strictly when making a prediction



Locally weighted Linear Regression

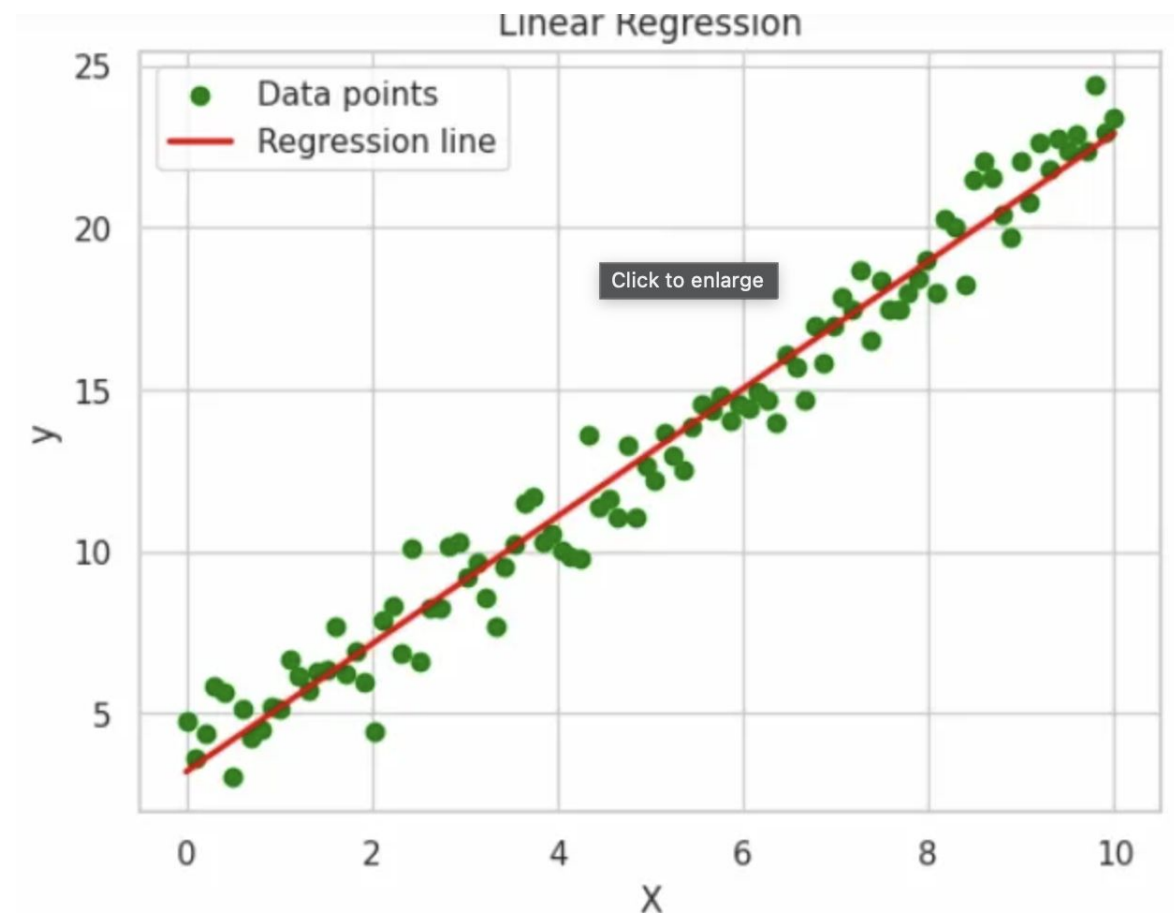
It is a widely used supervised learning algorithm that assumes a linear relationship exists between input X and output Y .

$$J(\theta) = \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$$

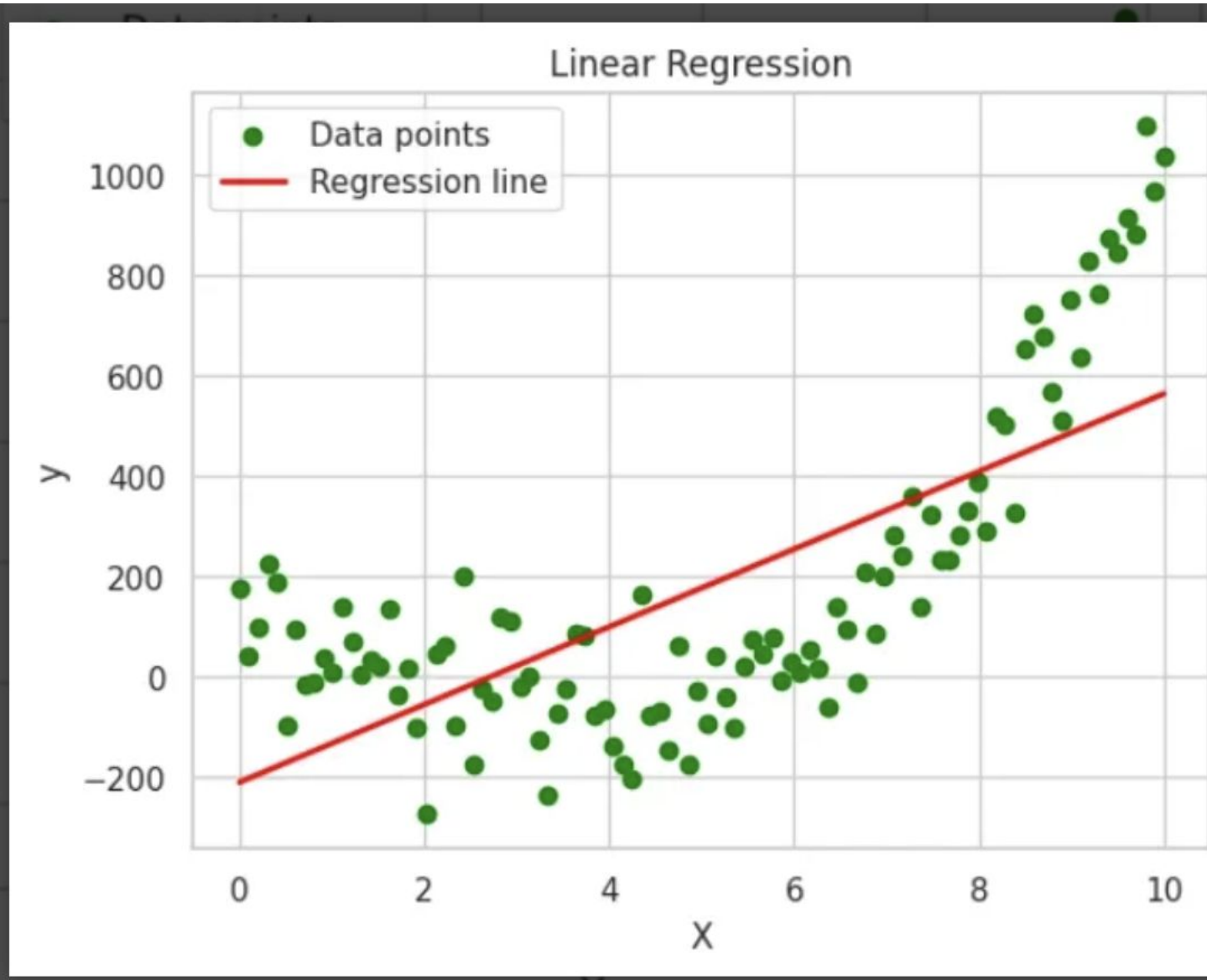
Where:

- $x^{(i)}$ is the feature vector of the i^{th} training example.
- $y^{(i)}$ is the corresponding output value of the i^{th} training example.
- θ is the vector of parameters (weights) that we want to optimize.





When the relationship between X and Y is non-linear, the standard linear regression model may not be effective. In such cases, Locally Weighted Linear Regression (LWLR) is used.



Locally Weighted Linear Regression

- Locally Weighted Linear Regression is a flexible method that adjusts the model to focus on the data points closest to each query point. Instead of creating one overall model like traditional linear regression, it creates a separate model for each prediction, using only the nearby data.
- This makes it useful when data behaves differently in different parts.
- The cost function is adjusted as follows:

Where:

$$J(\theta) = \sum_{i=1}^m w^{(i)} (\theta^T x^{(i)} - y^{(i)})^2$$

- $w^{(i)}$ is the weight associated with the i^{th} data point.
- The weight $w^{(i)}$ is calculated using a Gaussian kernel function and it decreases as the distance between $x^{(i)}$ and x increases.



Weight Calculation

The weight $\mathbf{w(i)}$ for each training point is computed using the following formula:

$$\mathbf{w(i)} = \exp(-((\mathbf{x(i)} - \mathbf{x})^2 / (2\tau^2)))$$

Where:

- τ = Bandwidth parameter that controls how quickly the weight decreases with distance.
- When $|\mathbf{x(i)} - \mathbf{x}|$ is small, the weight $\mathbf{w(i)}$ is close to **1**.
- When $|\mathbf{x(i)} - \mathbf{x}|$ is large, the weight $\mathbf{w(i)}$ approaches **0**.



Example

Consider the query point:

$$\mathbf{x} = 5.0$$

Training points:

$$\mathbf{x}(1) = 4.9$$

$$\mathbf{x}(2) = 3.0$$

Bandwidth:

$$\tau = 0.5$$

Weight for $\mathbf{x}(1)$

$$w(1) = \exp(-((4.9 - 5.0)^2 / (2 \times (0.5)^2)))$$

$$= \exp(-0.02)$$

$$= 0.9802$$

Weight for $\mathbf{x}(2)$

$$w(2) = \exp(-((3.0 - 5.0)^2 / (2 \times (0.5)^2)))$$

$$= \exp(-8)$$

$$= 0.000335$$

Since $\mathbf{x}(1) = 4.9$ is very close to $\mathbf{x} = 5.0$, it receives a **high weight (0.9802)**.

Since $\mathbf{x}(2) = 3.0$ is much farther away, it receives a **very small weight (0.000335)** and contributes very little to the prediction.



Key Features of LWLR

- Non-parametric: No global model, computes unique parameters for each query point.
- Local adaptation: Focuses on nearby data points for predictions.
- Weighted points: Closer points have more influence, distant points have less.
- Handles non-linearity: Great for complex, non-linear data relationships.
- No global parameters: Uses local fits instead of a single, global model.



Application

- **Non-linear Regression:** Models complex, non-linear relationships in data.
- **Time Series Forecasting:** Forecasts future values by focusing on recent data like stock prices, energy demand, etc.
- **Anomaly Detection:** Detects outliers by comparing local predictions with actual values like fraud detection.
- **Robotics:** Used in adaptive control and path planning for robots like self-driving cars.
- **Medical Data Analysis:** Predicts disease progression based on non-linear relationships in patient data like cancer progression.



Advantages of LWR

- **Flexibility:** Can model non-linear relationships by focusing on local data points.
- **Adaptivity:** Adjusts to local patterns, making it suitable for dynamic or non-stationary data.
- **Precision:** Provides better predictions for data with varying characteristics across different regions.
- **No Global Model:** Creates a new model for each prediction, making it highly customized to the query point.
- **Simple and Intuitive:** Easy to implement and understand compared to more complex models.



Limitations

- **Computationally Expensive:** Can be slow for large datasets due to the need to compute weights and solve for parameters for each query point.
- **Sensitive to Bandwidth (τ):** The performance heavily depends on the choice of the bandwidth parameter, requiring careful tuning.
- **No Global Model:** Lacks a single, unified model which can make it harder to interpret or generalize.
- **Overfitting Risk:** In areas with sparse data, the model may overfit to noise or anomalies.
- **Limited Scalability:** May not perform well on very large datasets without optimization techniques.



Radial Basis Function

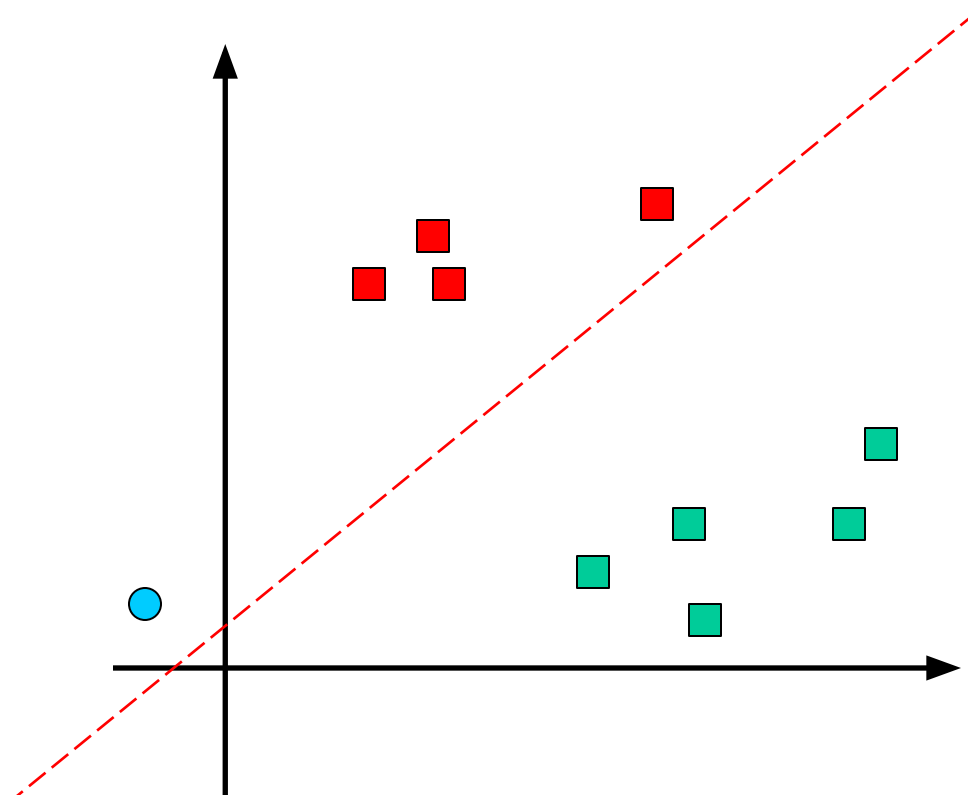
A **Radial Basis Function (RBF)** is a type of real-valued function whose value depends only on the distance from a center point. It is commonly used in **machine learning**, particularly in:

- Function approximation ,Classification, Time-series prediction
- Kernel methods like Support Vector Machines (SVMs)
- Radial Basis Function Networks (a type of neural network)

- General form $\phi(x) = \phi(\|x - c\|)$

- x : input point
- c : center
- $\|x - c\|$: distance between x and c , usually Euclidean
- ϕ : the radial basis function (e.g., Gaussian)

In MLP



Radial Basis Function Network

- A kind of supervised neural networks
- Design of NN as curve-fitting problem
- Learning
 - find surface in multidimensional space best fit to training data
- Generalization
 - Use of this multidimensional surface to interpolate the test data

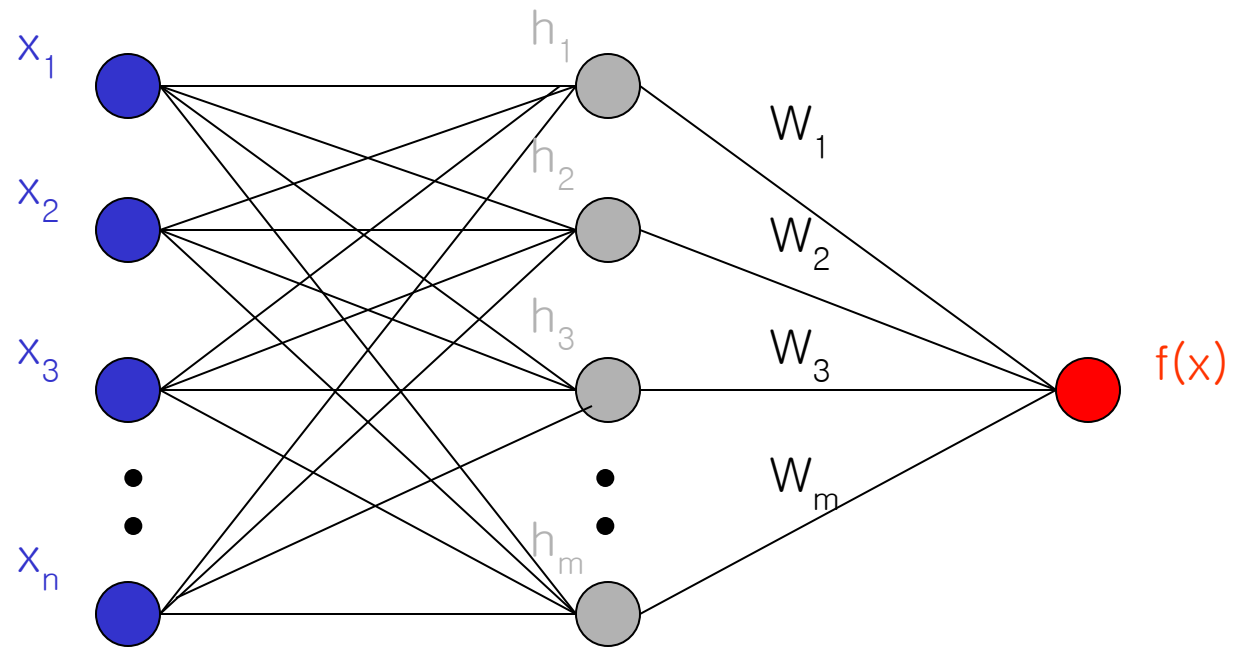
Radial Basis Function Network

- Approximate function with linear combination of Radial basis functions

$$F(x) = \sum w_i h(x)$$

- $h(x)$ is mostly Gaussian function

architecture



Three layers

- Input layer
 - Source nodes that connect to the network to its environment
- Hidden layer
 - Hidden units provide a set of basis function
 - High dimensionality
- Output layer
 - Linear combination of hidden functions

Radial basis function

$$f(x) = \sum_{j=1}^m w_j h_j(x)$$

$$h_j(x) = \exp(-(x-c_j)^2 / r_j^2)$$

Where c_j is center of a region,

r_j is width of the receptive field

designing

- Require
 - Selection of the radial basis function width parameter
 - Number of radial basis neurons

Selection of the RBF width para.

- Not required for an MLP
- smaller width
 - alerting in untrained test data
- Larger width
 - network of smaller size & faster execution

Number of radial basis neurons

- By designer
- Max of neurons = number of input
- Min of neurons = (experimentally determined)
- More neurons
 - More complex, but smaller tolerance

Various learning strategies

- How the centers of the radial-basis functions of the network are specified.
- Fixed centers selected at random
- Self-organized selection of centers
- Supervised selection of centers

Learning formula

- Linear weights (output layer)

$$\frac{\partial E(n)}{\partial w_i(n)} = \sum_{j=1}^N e_j(n) G(\|\mathbf{x}_j - \mathbf{t}_i(n)\|_{C_i}) \quad w_i(n+1) = w_i(n) - \eta_1 \frac{\partial E(n)}{\partial w_i(n)}, \quad i = 1, 2, \dots, M$$

- Positions of centers (hidden layer)

$$\frac{\partial E(n)}{\partial \mathbf{t}_i(n)} = 2w_i(n) \sum_{j=1}^N e_j(n) G'(\|\mathbf{x}_j - \mathbf{t}_i(n)\|_{C_i}) \Sigma_i^{-1} [\mathbf{x}_j - \mathbf{t}_i(n)] \quad \mathbf{t}_i(n+1) = \mathbf{t}_i(n) - \eta_2 \frac{\partial E(n)}{\partial \mathbf{t}_i(n)}, \quad i = 1, 2, \dots, M$$

- Spreads of centers (hidden layer)

$$\frac{\partial E(n)}{\partial \Sigma_i^{-1}(n)} = -w_i(n) \sum_{j=1}^N e_j(n) G'(\|\mathbf{x}_j - \mathbf{t}_i(n)\|_{C_i}) \mathbf{Q}_{ji}(n) \quad \mathbf{Q}_{ji}(n) = [\mathbf{x}_j - \mathbf{t}_i(n)][\mathbf{x}_j - \mathbf{t}_i(n)]^T$$

$$\Sigma_i^{-1}(n+1) = \Sigma_i^{-1}(n) - \eta_3 \frac{\partial E(n)}{\partial \Sigma_i^{-1}(n)}$$

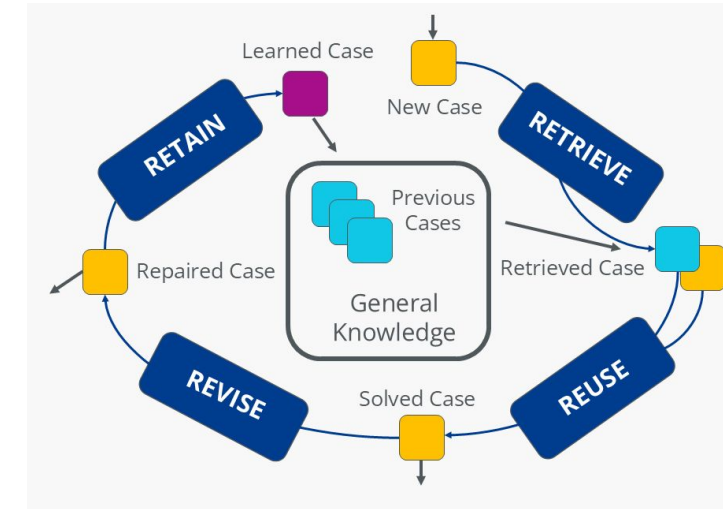
Case-Based Reasoning

- This approach uses past experiences to solve new but similar problems, relying on analogy.
- **Case:** A case is a problem that has been previously solved and stored in the database.
- **Similarity:** The similarity measure is used to determine the degree of resemblance between past cases and the current situation.
- **Adaptation:** Adaptation is the process of modifying a retrieved past case to fit the current situation.

Case-Based Reasoning Working

Case-based reasoning is a process that usually consists of four steps:

- 1. Retrieve:** The CBR system examines the case database to find experiences most similar to the current problem based on its description.
- 2. Reuse:** As a problem-solving approach, the solution of the case that is most similar to the description of the problem is used first. The first approach serves as the starting point for working on the new problem.
- 3. Revise:** The proposed solution is tested in the new context and refined as needed to fit specific conditions. Adjustments or corrections are made if required.
- 4. Retain:** The new problem-solving method is added to the case base for future requests. This creates an incremental learning process that ensures that the performance of the process increases with every case solved.



Practical Considerations for Optimizing Instance-Based Learning

- Memory Management
- Computational Efficiency
- Handling Noisy data
- Feature Scaling and Selection
- Adaptation to Evolving Data

Real World Applications of Instance-Based Learning

- Medical Diagnosis and Health care
- Fraud Detection in Financial Services
- Personalized Recommendations in E-commerce
- Image and Pattern Recognition
- Customer Support and Case Based Reasoning



Final Message

Success is not an overnight phenomenon.

It is the result of:

- Continuous learning
- Consistent daily practice
- Professional attitude
- Strong technical foundation

The best time to start was yesterday.

The second-best time is NOW.

Start preparing today for a brighter tomorrow!

GSFCU & Values

Respect & Courtesy



Dress
Appropriately



Respect
Personal Space



Be Mindful
of Your Words



Respect
Coworkers' Time



Keep Your
Workplace Clean



Socialise with
Your Peers



Use Your
Mobile Wisely



Take
Responsibility

➤➤ **Thank You..**